

Iterated Local Search (ILS) para a otimização do Índice de Qualidade Caminhável (IQC): método clássico, implementação e análise dos resultados

1 ILS

Esta seção descreve o Iterated Local Search na sua forma clássica, seguindo a apresentação de Lourenço, Martin e Stützle [1]. O objetivo é registrar o método como ele foi proposto, sem ainda tratar do problema desta tese. As decisões de adaptação ficam para a Seção 2, e a análise dos resultados obtidos com os dados do projeto, para a Seção 3.

1.1 Visão geral

O ILS é uma metaheurística que trabalha sobre um algoritmo de busca local já existente, tratado como uma rotina que recebe uma solução e devolve um ótimo local. A ideia central é construir uma sequência de soluções geradas por essa busca local, de modo que a sequência leve a soluções bem melhores do que as obtidas por tentativas aleatórias independentes. Dois pontos definem o método: existe uma única cadeia de soluções sendo percorrida, o que exclui métodos baseados em população; e a busca por soluções melhores ocorre em um espaço reduzido, definido pela saída da busca local.

Para formalizar, seja f a função de custo de um problema de otimização combinatória, a ser minimizada, e seja S o conjunto das soluções. A busca local, aqui chamada de **LocalSearch**, mapeia uma solução qualquer para um ótimo local, ou seja, define uma aplicação de S no conjunto menor S^* dos ótimos locais. O ILS percorre uma caminhada dentro de S^* : a partir de um ótimo local corrente, aplica uma perturbação, roda a busca local sobre o resultado, chega a um novo ótimo local e decide, por um critério de aceitação, se aceita ou não o novo ponto. A força do método está nessa amostragem enviesada do conjunto dos ótimos locais, que é muito melhor do que a amostragem aleatória.

O motivo pelo qual isso funciona aparece quando se compara a distribuição dos custos. Se sortearmos soluções ao acaso em S e ótimos locais em S^* , a distribuição dos custos dos ótimos locais tem média e variância menores.

Algorithm 1 Iterated Local Search (minimização)

```
1:  $s_0 \leftarrow \text{GENERATEINITIALSOLUTION}$ 
2:  $s^* \leftarrow \text{LOCALSEARCH}(s_0)$ 
3: repeat
4:    $s' \leftarrow \text{PERTURBATION}(s^*, \text{history})$ 
5:    $s^{*'} \leftarrow \text{LOCALSEARCH}(s')$ 
6:    $s^* \leftarrow \text{ACCEPTANCECRITERION}(s^*, s^{*'}, \text{history})$ 
7: until condição de parada satisfeita
```

Portanto vale a pena usar a busca local em vez de amostrar S diretamente. Em instâncias grandes, porém, a distribuição dos custos dos ótimos locais fica muito concentrada em torno da média, e reiniciar a busca local de pontos aleatórios (o chamado *random restart*) passa a ter chance cada vez menor de encontrar as soluções muito boas, que existem mas são raras. Para alcançá-las é preciso uma amostragem enviesada, e é isso que o ILS faz ao caminhar de um ótimo local para outro próximo em vez de recomeçar do zero.

1.2 Os quatro componentes

O ILS é descrito por quatro componentes, e a arquitetura de alto nível aparece no Algoritmo 1. **GenerateInitialSolution** produz a solução inicial. **LocalSearch** leva uma solução a um ótimo local. **Perturbation** modifica o ótimo local corrente, produzindo um ponto intermediário. **AcceptanceCriterion** decide se o novo ótimo local passa a ser o corrente. O termo *history* indica que perturbação e aceitação podem depender do histórico da busca.

Linha a linha do Algoritmo 1: a linha 1 gera a solução inicial; a linha 2 leva essa solução a um ótimo local, que é o ponto de partida da caminhada; o laço das linhas 3 a 7 repete a perturbação (linha 4), a busca local sobre o ponto perturbado (linha 5) e o teste de aceitação (linha 6) até a parada. Se a perturbação e a aceitação não dependem do histórico, a caminhada não tem memória e é markoviana: a probabilidade de ir de um ótimo local a outro depende só desses dois pontos. A maior parte dos trabalhos com ILS é desse tipo, embora o uso de memória possa melhorar o desempenho.

Uma observação prática do método: é possível desenvolver uma versão básica de ILS com pouco esforço. Basta começar de uma solução aleatória ou de uma construção gulosa; usar uma busca local disponível; usar como perturbação um movimento aleatório em uma vizinhança de ordem maior do que a da busca local; e, como aceitação, exigir que o custo diminua. Essa versão básica já costuma superar o random restart, e cada um dos quatro componentes pode depois ser refinado de forma modular.

1.3 Solução inicial

A busca local aplicada à solução inicial define o ponto de partida da caminhada. As escolhas usuais são uma solução aleatória ou uma solução de uma construção gulosa. A construção gulosa tem duas vantagens: combinada com a busca local, tende a produzir soluções finais melhores, e a busca local a partir dela costuma exigir menos passos, portanto menos tempo. A importância da solução inicial depende do tempo de execução: para execuções curtas ela pesa mais; para execuções longas, o efeito da solução inicial se dissipa se a caminhada explora bem o espaço dos ótimos locais. Quando o efeito da solução inicial persiste mesmo em execuções longas, isso é sinal de que a caminhada está tendo dificuldade em explorar, e é preciso rever perturbação e aceitação.

1.4 Perturbação

A limitação da descida local é ficar presa em ótimos locais piores que o global. O ILS escapa desses ótimos aplicando perturbações. A força de uma perturbação é o número de componentes da solução que ela modifica. Em geral, a busca local não deve conseguir desfazer a perturbação, senão a caminhada volta ao mesmo ótimo local. Um movimento aleatório em uma vizinhança de ordem maior do que a usada pela busca local costuma ser suficiente para isso.

O tamanho da perturbação precisa de equilíbrio. Se for grande demais, o ILS passa a se comportar como random restart, e as soluções boas só aparecem com probabilidade muito baixa. Se for pequena demais, a busca local volta com frequência ao ótimo local recém-visitado, e a diversificação fica limitada. Para alguns problemas, uma força pequena e fixa basta; para outros, é preciso força maior, e a melhor força depende da instância. Isso motiva as perturbações adaptativas, em que a força é modificada durante a execução. Uma forma de fazer isso é explorar o histórico: aumentar a força quando a busca estaciona e reduzi-la quando volta a melhorar. A busca em vizinhança variável básica (VNS básico) é um exemplo de esquema que varia a força de forma sistemática. Vale ainda notar que a força da perturbação afeta a velocidade da busca local: perturbações fracas em geral levam a execuções mais rápidas da busca local, o que permite ao ILS percorrer muito mais ótimos locais do que o random restart no mesmo tempo.

1.5 Critério de aceitação

O critério de aceitação controla o equilíbrio entre intensificação e diversificação da caminhada em S^* . Dois casos extremos ilustram isso. O critério

Better aceita apenas soluções melhores, o que é uma intensificação forte:

$$\text{Better}(s^*, s^{*'}) = \begin{cases} s^{*'} & \text{se } f(s^{*'}) < f(s^*), \\ s^* & \text{caso contrário.} \end{cases} \quad (1)$$

No extremo oposto está o passeio aleatório **RW**, que sempre aceita o novo ótimo local, favorecendo a diversificação:

$$\text{RW}(s^*, s^{*'}) = s^{*'} \quad (2)$$

Entre os dois há muitas escolhas intermediárias. O critério **LSMC**, do tipo recozimento simulado, aceita $s^{*'}$ sempre que é melhor e, quando é pior, aceita com probabilidade $\exp((f(s^*) - f(s^{*'}))/T)$, com T uma temperatura. O critério **Restart** reinicia a busca de uma nova solução quando nenhuma melhora é encontrada por um número dado de iterações. Há forte interdependência entre a força da perturbação e o critério de aceitação, e uma regra prática do método é que, quando é preciso diversificar, costuma ser melhor aceitar várias perturbações pequenas do que uma perturbação grande.

1.6 Busca local e otimização global

A busca local é tratada como uma caixa-preta chamada muitas vezes. Em geral, quanto melhor a busca local, melhor o ILS correspondente, mas há um compromisso com o tempo: às vezes compensa usar uma busca local mais rápida e menos poderosa se ela pode ser aplicada muito mais vezes no mesmo tempo total. A varredura pode ser por primeira melhora ou por melhor melhora. Um cuidado importante é que a busca local não deve desfazer sistematicamente a perturbação. A otimização global do ILS é, na prática, um processo de refinar cada componente considerando os outros, porque a melhor perturbação depende da busca local e o melhor critério de aceitação depende das duas. O que se busca otimizar costuma ser a média, sobre execuções, do melhor custo encontrado em uma execução de duração fixa, e é preferível não ajustar o algoritmo a ponto de torná-lo sensível aos detalhes de uma instância específica.

Em relação a outras metaheurísticas, o VNS básico pode ser visto como um ILS que usa o critério **Better** e uma forma sistemática de variar a força da perturbação. A diferença de filosofia é que o ILS tem como objetivo explícito construir uma caminhada no conjunto dos ótimos locais, enquanto o VNS parte da ideia de trocar de vizinhança de forma sistemática.

2 ILS no projeto

Esta seção relaciona o método da Seção 1 com a implementação, feita no módulo `metaheuristics/methods/ils.py`. A cada componente da teoria corresponde uma decisão de código, e essas correspondências são explicitadas ao longo do texto.

2.1 O problema e a representação da solução

O projeto calcula, para cada hexágono H3 de uma região, onze indicadores de caminhabilidade e, a partir deles, o Índice de Qualidade Caminhável (IQC), com pesos obtidos pelo método CRITIC. A otimização aloca um orçamento fixo de pontos de interesse (POIs) para maximizar a soma do IQC sobre todos os hexágonos. Os POIs entram em sete dimensões (**S_saude**, **S_educacao**, **S_abastecimento**, **S_lazer**, **S_servicos**, **T_transporte**, **U_urbanidade**). Um POI inserido em um hexágono de origem propaga impacto para hexágonos de destino segundo um peso α_{20} , derivado do tempo de caminhada por um decaimento em cosseno. O orçamento é de 100 POIs e a linha de base é a soma do IQC sem nenhuma alocação.

Uma solução é uma matriz de inteiros não negativos, com uma linha por hexágono e uma coluna por dimensão de POI, cujos valores somam o orçamento e são não nulos apenas nas linhas de origem elegíveis. A função objetivo constrói a matriz final de indicadores a partir da linha de base e do impacto propagado,

$$x_{t,c}^{\text{final}} = x_{t,c}^{\text{base}} + \sum_h \alpha_{20}(h, t) q_{h,c}, \quad (3)$$

onde $q_{h,c}$ é a quantidade de POIs na origem h e dimensão c ; em seguida recalcula os pesos CRITIC e o IQC, e devolve a soma

$$f(S) = \sum_h \text{IQC}_h, \quad (4)$$

a ser maximizada. Essa é exatamente a mesma função objetivo usada pelos demais métodos do projeto.

2.2 Os quatro componentes no projeto

O mapeamento dos quatro componentes do ILS é o seguinte. **GenerateInitialSolution** é uma alocação espacial aleatória do orçamento entre pares (origem elegível, dimensão), uma por semente. **LocalSearch** é uma busca por primeira melhora com movimento de realocação que preserva a dimensão. **Perturbation** é uma realocação em blocos que também preserva a dimensão, com força adaptativa. **AcceptanceCriterion** é o critério **Better**, adaptado para maximização. Uma consequência de preservar a dimensão tanto na busca local quanto na perturbação é que a distribuição de POIs por dimensão fica fixada pela solução inicial de cada semente; a busca move POIs entre hexágonos dentro de cada dimensão, mas não entre dimensões.

2.3 Glossário de variáveis

A Tabela 1 reúne os símbolos, os nomes no código e o significado de cada variável usada nos algoritmos.

Tabela 1: Variáveis usadas na formulação e no código.

Símbolo	Nome no código	Significado
$f(S)$	<code>objective_value</code>	soma do IQC da solução
B	<code>budget</code>	orçamento; número de POIs (100)
$q_{h,d}$	<code>candidate_matrix</code>	solução; POIs na origem h , dimensão d
$x^{\text{base}}, x^{\text{final}}$	<code>baseline_matrix</code> , <code>final_matrix</code>	indicadores antes e depois do impacto
$\alpha_{20}(h, t)$	<code>alpha_values</code>	peso de impacto do par origem/destino
—	<code>allowed_source_rows</code>	hexágonos elegíveis a receber POI
s_0	<code>initial_candidate_matrix</code>	solução inicial (alocação aleatória)
s^*	<code>current_candidate_matrix</code>	ótimo local corrente da caminhada
—	<code>best_candidate_matrix</code>	melhor solução da execução
—	<code>current_objective_value</code>	valor de s^*
—	<code>best_objective_value</code>	valor da melhor solução
—	movimento (sr, sc, tr, tc)	realocar 1 POI de (sr, sc) para (tr, tc)
—	<code>perturbation_strength</code>	força da perturbação (nº de blocos)
—	<code>perturbation_block_size</code>	POIs movidos por bloco
—	<code>adaptive_perturbation</code>	se a força varia durante a execução
—	<code>no_improve_streak</code>	iterações seguidas sem melhorar a melhor solução
—	<code>local_search_neighbor_sample</code>	nº de vizinhos amostrados por passo de busca local
—	<code>local_search_max_steps</code>	passos máximos de busca local
—	<code>evaluations</code>	avaliações reais da função objetivo
—	<code>iterations</code>	iterações ILS da execução
—	<code>max_evaluations</code>	teto de avaliações (30 000)
—	<code>max_iterations</code>	teto de iterações (500)
—	<code>max_no_improve</code>	teto de iterações sem melhora (80)
—	<code>random_generator</code>	gerador pseudoaleatório da semente

2.4 Solução inicial e busca local linha a linha

A solução inicial de cada semente é uma alocação espacial aleatória: para cada um dos 100 POIs, sorteiam-se uma origem elegível e uma dimensão, de forma determinística a partir da semente. Sobre essa solução aplica-se a busca local, que é o mesmo operador reutilizado pelo GRASP no projeto.

Ele tem duas partes: o sorteio de um movimento de realocação e o laço de primeira melhora.

Listing 1: Sorteio de um movimento de realocação viável.

```

1 def _sample_feasible_relocation_move(cand, rng, allowed_rows,
2                                     preserve_dimension=True):
3     src_positions = np.argwhere(cand[allowed_rows, :] > 0)
4     if src_positions.size == 0:
5         return None
6     n_cols = cand.shape[1]
7     n_rows = allowed_rows.shape[0]
8     for _ in range(12):
9         i = int(rng.integers(0, len(src_positions)))
10        src_row_local, src_col = src_positions[i]
11        src_row = int(allowed_rows[src_row_local])
12        tgt_row = int(allowed_rows[rng.integers(0, n_rows)])
13        tgt_col = int(src_col) if preserve_dimension else int(rng.
14        integers(0, n_cols))
15        if src_row == tgt_row and src_col == tgt_col:
16            continue
17        return src_row, src_col, tgt_row, tgt_col
18    return None

```

Na Listagem 1, a linha 3 lista as posições da solução, restritas às linhas elegíveis, que já têm ao menos um POI; são as origens possíveis. As linhas 4 e 5 abortam se não houver nenhuma. O laço da linha 8 tenta até doze vezes montar um movimento válido. A linha 9 sorteia uma origem; as linhas 10 e 11 recuperam sua linha e coluna. A linha 12 sorteia a linha de destino entre as elegíveis. A linha 13 fixa a coluna de destino igual à de origem quando `preserve_dimension` é verdadeiro, que é o modo usado. As linhas 14 e 15 descartam o movimento nulo. A linha 16 devolve o movimento como a quádrupla (linha de origem, coluna de origem, linha de destino, coluna de destino).

Listing 2: Busca local por primeira melhora.

```

1 def _run_local_search_first_improvement(cand, state, rng, value,
2     max_steps, neighbor_sample, allowed_rows, eps,
3     preserve_dimension=True):
4     accepted = 0
5     for _ in range(max_steps):
6         improved = False
7         for _ in range(neighbor_sample):
8             move = _sample_feasible_relocation_move(
9                 cand, rng, allowed_rows, preserve_dimension)
10            if move is None:
11                break
12            sr, sc, tr, tc = move
13            cand[sr, sc] -= 1.0
14            cand[tr, tc] += 1.0
15            trial = objective_function(

```

```

16         build_final_indicator_matrix_nd(cand, state))["
    objective_value"]
17         if trial > value + eps:
18             value = trial
19             accepted += 1
20             improved = True
21             break
22         cand[tr, tc] -= 1.0
23         cand[sr, sc] += 1.0
24         if not improved:
25             break
26     return value, accepted

```

Na Listagem 2, o laço externo da linha 5 executa até `max_steps` passos de melhora. A cada passo, o laço interno da linha 7 sorteia até `neighbor_sample` vizinhos. A linha 8 obtém um movimento; sem movimento viável, a linha 11 interrompe. As linhas 13 e 14 aplicam o movimento. As linhas 15 e 16 avaliam a solução alterada pela função objetivo verdadeira, o que conta como uma avaliação. A linha 17 testa a melhora acima de uma tolerância. Havendo melhora, as linhas 18 a 21 aceitam o movimento e saem do laço interno, o que caracteriza a primeira melhora: aceita-se o primeiro vizinho que melhora, sem varrer os demais. Sem melhora, as linhas 22 e 23 desfazem o movimento. Quando um passo inteiro não encontra melhora, a linha 25 encerra a busca. Como a vizinhança completa é grande, o operador usa amostragem, uma estratégia de vizinhança restrita.

2.5 Perturbação linha a linha

A perturbação é uma realocação em blocos que preserva a dimensão. A força é o número de blocos, e cada bloco move um número fixo de POIs (`block_size`).

Listing 3: Perturbação em blocos que preserva a dimensão.

```

1 def _apply_dimension_preserving_block_perturbation(cand, rng, strength,
2                                                     allowed_rows,
3
4             block_size=2):
5     applied = 0
6     last_move = None
7     for _ in range(strength):
8         block_applied = 0
9         attempts = 0
10        while block_applied < block_size and attempts < max(12,
11            block_size * 12):
12            attempts += 1
13            move = _sample_feasible_relocation_move(
14                cand, rng, allowed_rows, preserve_dimension=True)
15            if move is None:
16                break
17            sr, sc, tr, tc = move

```



```

15         if (last_move is not None and sr == last_move[2] and tr ==
            last_move[0]
16             and sc == last_move[1] and tc == last_move[3]):
17             continue
18             cand[sr, sc] -= 1.0
19             cand[tr, tc] += 1.0
20             applied += 1
21             block_applied += 1
22             last_move = move
23         if block_applied == 0:
24             break
25     return applied

```

Na Listagem 3, o laço da linha 5 repete-se uma vez por bloco, ou seja, **strength** vezes. O laço interno da linha 8 aplica movimentos até completar o bloco (**block_size** movimentos) ou esgotar as tentativas. A linha 10 sorteia um movimento de realocação preservando a dimensão. A linha 14 descarta o movimento que seria o inverso imediato do anterior, o que evita ciclos curtos, um cuidado recomendado pela teoria para perturbações que a busca local poderia desfazer. As linhas 16 e 17 aplicam o movimento; as linhas 18 a 20 atualizam os contadores. A linha 23 interrompe se um bloco não conseguiu aplicar nenhum movimento. Preservar a dimensão e mover mais de um POI por bloco cria uma perturbação de ordem maior do que o movimento único da busca local, de modo que a busca local não a desfaz com um só passo; isso atende à recomendação de que a busca local não deve desfazer a perturbação.

2.6 Aceitação, força adaptativa e o laço por semente

O critério de aceitação é o **Better** da Equação (1), adaptado para maximização: aceita-se a solução após a busca local se seu valor supera o corrente. Sob esse critério a solução corrente só melhora, e a perturbação é sempre aplicada à melhor solução da caminhada. A força da perturbação é adaptativa: ao melhorar a melhor solução, a força volta ao valor base; ao estagnar, a força aumenta em uma unidade a cada bloco fixo de iterações sem melhora, até um teto. Esse esquema segue a ideia de perturbação adaptativa descrita na teoria, que aumenta a diversificação quando a intensificação estaciona. O Algoritmo 2 descreve o procedimento implementado, para uma semente.

Linha a linha do Algoritmo 2: a linha 3 gera a solução inicial; a linha 4 aplica a busca local, definindo o ponto de partida da caminhada; a linha 5 inicializa contadores e a força. O laço da linha 6 continua enquanto nenhum dos três limites for atingido. A linha 8 perturba a solução corrente; a linha 9 aplica a busca local sobre o ponto perturbado. A linha 10 testa a aceitação **Better**: se o novo ótimo local é melhor, as linhas 11 e 12 o adotam como corrente e melhor, zeram o contador de estagnação e devolvem a força ao valor base. Caso contrário, a linha 14 incrementa a estagnação e as linhas 15 e 16 aumentam a força quando a estagnação atinge um múltiplo do intervalo de

Algorithm 2 ILS implementado, por semente (maximização de $\sum \text{IQC}$)

```
1: procedure ILS-SEMENTE(mente)
2:   inicialize o gerador pseudoaleatório com mente
3:    $s_0 \leftarrow$  alocação espacial aleatória do orçamento
4:    $s^* \leftarrow$  BuscaLocalPrimeiraMelhora( $s_0$ );  $f^* \leftarrow f(s^*)$ 
5:   melhor  $\leftarrow s^*$ ; avaliacoes, iteracoes, sem_melhora  $\leftarrow 0$ ; força  $\leftarrow$ 
     base
6:   while avaliacoes < 30000 e iteracoes < 500 e sem_melhora < 80 do
7:     iteracoes  $\leftarrow$  iteracoes + 1
8:      $s' \leftarrow$  Perturbação( $s^*$ , força) ▷ Listagem 3
9:      $s'^* \leftarrow$  BuscaLocalPrimeiraMelhora( $s'$ ) ▷ Listagem 2
10:    if  $f(s'^*) > f(s^*)$  then ▷ aceitação Better
11:       $s^* \leftarrow s'^*$ ;  $f^* \leftarrow f(s'^*)$ ; melhor  $\leftarrow s^*$ 
12:      sem_melhora  $\leftarrow 0$ ; força  $\leftarrow$  base
13:    else
14:      sem_melhora  $\leftarrow$  sem_melhora + 1
15:      if adaptativa e sem_melhora múltiplo de 10 then
16:        força  $\leftarrow$  min(força + 1, teto)
17:      end if
18:    end if
19:  end while
20:  return melhor,  $f^*$  e as métricas da execução
21: end procedure
```

adaptação. A linha 20 devolve a melhor solução e as métricas. As execuções por semente são independentes e rodam em processos paralelos.

2.7 Fluxo de execução

A Figura 1 resume o fluxo de uma execução por semente, e a Figura 2 mostra como o orçamento de avaliações se distribui dentro de uma iteração. A perturbação não consome avaliações; a avaliação da solução perturbada consome uma; e a busca local consome uma avaliação por vizinho testado, até o limite de $25 \times 64 = 1\,600$.

2.8 Sementes, orçamento e parâmetros

As sementes são lidas de `seeds.txt` (100 inteiros); cada semente é uma execução completa e independente, com seu próprio gerador pseudoaleatório, que governa a alocação inicial, a amostragem de vizinhos na busca local e o sorteio de movimentos na perturbação. O critério de parada usa três limites: 30 000 avaliações da função objetivo, 500 iterações e 80 iterações sem melhora da melhor solução. A Tabela 2 lista os parâmetros e as variáveis de ambiente

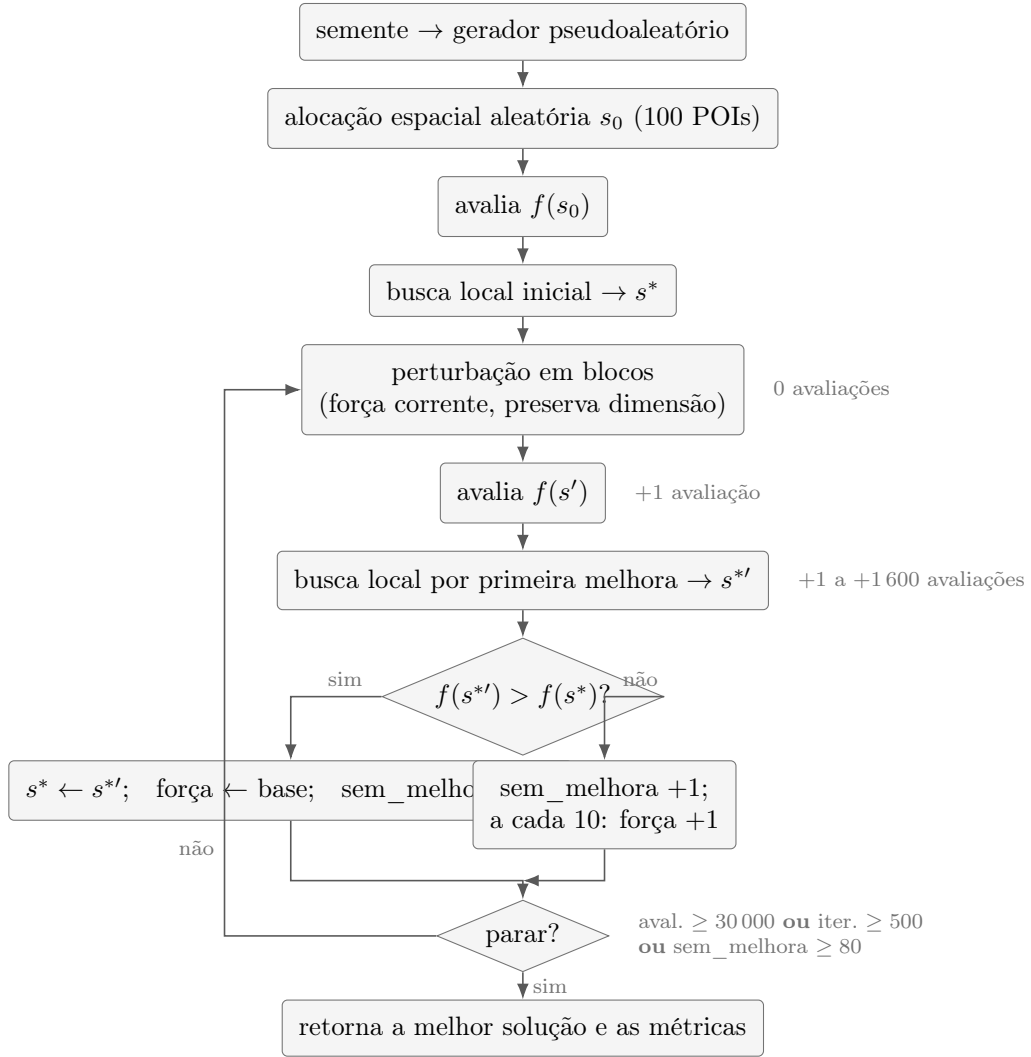


Figura 1: Fluxo de uma execução ILS por semente, como implementado. A caminhada mantém um único ótimo local corrente s^* , perturbado e reotimizado a cada iteração; a aceitação **Better** e a força adaptativa controlam intensificação e diversificação.



Figura 2: Anatomia de uma iteração ILS sob a ótica do orçamento. O custo em avaliações concentra-se na busca local; a fração de tempo vem dos registros de perfil das execuções.

que os controlam.

Tabela 2: Parâmetros da implementação do ILS.

Parâmetro	Padrão	Variável de ambiente
Máximo de avaliações	30 000	—
Máximo de iterações	500	—
Máximo sem melhora	80	—
Força de perturbação (base)	5	ILS_PERTURBATION_STRENGTH
Tamanho do bloco	2	ILS_PERTURBATION_BLOCK_SIZE
Perturbação adaptativa	sim	ILS_ADAPTIVE_PERTURBATION
Intervalo de adaptação	10	ILS_PERTURBATION_ADAPT_EVERY
Amostra de vizinhos	64	—
Passos máximos de busca local	25	—
Critério de aceitação	better	—

O valor base da força de perturbação no código é 5. Os resultados analisados na Seção 3 foram gerados com dois valores dessa força, 2 e 10, justamente para estudar o efeito desse parâmetro.

2.9 Discussão dos valores dos parâmetros

Os valores da Tabela 2 não foram escolhidos de forma arbitrária. Cada um tem uma justificativa ligada à teoria da Seção 1, ao custo computacional do problema ou ao desenho da comparação com os demais métodos do projeto. Esta subseção percorre essas justificativas.

O teto de 30 000 avaliações define o esforço computacional em número de chamadas da função objetivo, e não em tempo. A escolha da unidade tem dois motivos. Primeiro, a avaliação é o custo dominante da execução: cada chamada recalcula os pesos CRITIC e o IQC sobre a matriz inteira, e os registros de perfil das execuções mostram que essas chamadas respondem, em média, por 94% do tempo total (entre 90 e 97% conforme a instância). Segundo, um teto em avaliações não depende da máquina nem do paralelismo, e o mesmo teto é adotado pelo GRASP, o que define a comparação entre métodos sob esforço idêntico. Quanto à ordem de grandeza: uma única chamada da busca local pode consumir até $25 \times 64 = 1\,600$ avaliações no pior caso, de modo que 30 000 avaliações acomodam da ordem de uma centena de iterações ILS por semente; as execuções da Seção 3 fizeram em média entre 106 e 187 iterações. Além disso, a análise de convergência (Tabela 7) mostra que cerca de 90% do ganho é obtido até por volta de 5 000 avaliações, então o teto não corta a busca em uma região de ganho relevante.

O limite de 500 iterações é uma salvaguarda para o caso de iterações anormalmente baratas, em que o teto de avaliações demoraria a ser atingido. Na prática ele não foi ativado: entre as 3 000 execuções analisadas, todas pararam por orçamento de avaliações ou por estagnação (Tabela 6), nenhuma por número de iterações.

O limite de 80 iterações sem melhora define o que se considera estagnação, e seu valor está acoplado ao intervalo de adaptação da perturbação. Como a força aumenta uma unidade a cada 10 iterações sem melhora, a execução tenta até oito níveis crescentes de diversificação antes de desistir. Um limite muito menor cortaria a execução antes de a adaptação ter chance de agir; um limite muito maior gastaria orçamento em uma caminhada que já não progride.

Os dois parâmetros da busca local controlam o custo de cada chamada. A vizinhança completa do movimento de realocação tem tamanho da ordem do produto entre as posições não nulas da solução (até 100) e as origens elegíveis (entre 167 e 194 nas instâncias do projeto), o que dá dezenas de milhares de vizinhos; uma varredura completa por melhor melhora gastaria boa parte do orçamento em um único passo. A amostragem de 64 vizinhos com primeira melhora é a estratégia de vizinhança restrita discutida na teoria, e o limite de 25 passos limita o custo de uma chamada a 1 600 avaliações, cerca de 5% do orçamento, impedindo que uma única busca local consuma a execução. A primeira melhora foi preferida à melhor melhora por custar menos avaliações por movimento aceito.

O tamanho de bloco 2 da perturbação faz dela um movimento de ordem maior do que o da busca local, que realoca um único POI por vez: desfazer um bloco exige dois movimentos coordenados na mesma dimensão, o que torna improvável que a busca local desfaça a perturbação e devolva a caminhada ao mesmo ótimo local, que é a exigência central da teoria sobre perturbações. A força da perturbação é o número de blocos; com bloco 2, uma perturbação de força s move até $2s$ dos 100 POIs da solução, ou seja, cerca de 4% da solução com $s = 2$ e 20% com $s = 10$. Esses dois valores foram usados nos experimentos por estarem em lados opostos do compromisso descrito na teoria — perturbação fraca demais devolve a busca ao mesmo ótimo, forte demais aproxima o método do random restart — e a análise da Seção 3 mostra que a força menor foi melhor em todas as instâncias, com a maior levando à estagnação precoce. O valor base 5 do código fica no meio desse intervalo. A adaptação, com aumento de um nível a cada 10 iterações sem melhora e retorno ao valor base quando há melhora, implementa a recomendação de perturbações adaptativas: diversifica quando a intensificação estaciona e volta ao regime fraco assim que a busca reencontra progresso.

O critério de aceitação **better** é o extremo de intensificação da Equação (1). A escolha leva em conta o orçamento limitado: critérios mais diversificadores, como o passeio aleatório ou o de tipo recozimento, exploram mais o espaço dos ótimos locais, mas pagam por isso em avaliações, e a

adaptação da força de perturbação já provê diversificação quando a busca estaciona. A tolerância numérica de 10^{-12} existe apenas para que ruído de ponto flutuante não seja tratado como melhora: como o IQC é arredondado a quatro casas decimais, qualquer melhora real da soma é da ordem de 10^{-4} ou maior, muitas ordens de grandeza acima da tolerância.

Por fim, as 100 sementes por instância definem o tamanho amostral da análise. Com $n = 100$, o intervalo de confiança de 95% para a média tem meia-largura de aproximadamente 0,2 desvio padrão amostral, estreito o suficiente para as comparações da Seção 3. O uso das mesmas sementes em todas as configurações e métodos é o que habilita as comparações pareadas, que têm mais poder estatístico do que as não pareadas.

2.10 Exemplo de execução passo a passo

Este exemplo executa os operadores do ILS sobre uma instância pequena, com quatro hexágonos h_0 a h_3 , duas dimensões de POI (**S_saude** e **S_lazer**) e um terceiro indicador fixo (**I_seguranca**), com apenas h_0 e h_1 elegíveis e orçamento de três POIs. Para focar a caminhada em uma dimensão, considere as alocações na dimensão **S_saude**. Como a busca local preserva a dimensão e há duas origens elegíveis, as soluções alcançáveis são os quatro pares (h_0, h_1) de contagens que somam três. A Tabela 3 dá o valor da função objetivo de cada uma. A linha de base (sem POIs) vale $f = 2,0000$.

Tabela 3: Objetivo das soluções alcançáveis em **S_saude** (orçamento 3).

(h_0, h_1) em S_saude	(3, 0)	(2, 1)	(1, 2)	(0, 3)
f	1,8852	1,8605	2,0155	1,7397

A solução inicial da semente coloca os três POIs em h_1 , ou seja, (0, 3), com $f = 1,7397$. A busca local, por primeira melhora, realoca uma unidade de h_1 para h_0 , chega a (1, 2) com $f = 2,0155$ e para, porque nenhum outro movimento único melhora ((2, 1) vale 1,8605 e (0, 3) é pior). Esse é o ponto de partida s^* da caminhada, já acima da linha de base.

Na iteração seguinte, a perturbação realoca uma unidade dentro de **S_saude**, levando a solução de (1, 2) de volta a (0, 3), com $f = 1,7397$. A busca local então sobe outra vez para (1, 2), com $f = 2,0155$. Pelo critério **Better**, o novo ótimo local não supera o corrente (2,0155 não é maior que 2,0155), então a perturbação é rejeitada e a caminhada permanece em (1, 2). Neste exemplo mínimo há um único ótimo local alcançável em **S_saude**, de modo que a perturbação retorna a ele; em instâncias reais, com muitos hexágonos, existem vários ótimos locais, e a perturbação permite alcançar ótimos diferentes, às vezes melhores. É esse o mecanismo que, ao longo de muitas iterações, produz os ganhos medidos na Seção 3.

3 Análise dos resultados

Esta seção analisa os resultados produzidos pela implementação do ILS. Todos os números vêm dos arquivos gravados em `results/ils/`.

3.1 Desenho experimental

Os experimentos cobrem cinco localidades (`av_paulista`, `itajuba_centro`, `metro_ana_rosa`, `milao_italia`, `shinjuku_toquio`) e três perfis de caminhada (`athlete`, `average_adult`, `elderly`), totalizando quinze instâncias. Cada instância foi resolvida com 100 sementes, e cada semente é uma execução completa e independente. Todas as execuções usaram o mesmo teto de 30 000 avaliações, o critério de aceitação `better`, a perturbação em blocos que preserva a dimensão (tamanho de bloco 2) e a força adaptativa. Cada instância foi executada em duas configurações de força base de perturbação, 2 e 10, com as mesmas 100 sementes, o que permite uma comparação pareada entre as duas forças. No total são 30 execuções de 100 sementes, ou seja, 3 000 execuções independentes.

3.2 Metodologia estatística

Para cada instância e configuração, as 100 sementes são tratadas como uma amostra de 100 observações independentes do valor final produzido pelo algoritmo. Reporta-se a média, o desvio padrão amostral (com $n - 1$ no denominador), a mediana, o mínimo, o máximo, o intervalo interquartilico e um intervalo de confiança de 95% para a média, calculado por $\bar{x} \pm t_{0,975;99} s / \sqrt{n}$, com $t_{0,975;99} \approx 1,984$. O coeficiente de variação (desvio padrão sobre média, em porcentagem) mede a dispersão relativa. A melhoria sobre a linha de base é medida em porcentagem, $100 (f - f_{\text{base}}) / f_{\text{base}}$.

A comparação entre as forças de perturbação 2 e 10 é pareada: como as duas configurações usaram as mesmas sementes, compara-se, semente a semente, a diferença entre os valores finais. Aplica-se o teste de postos sinalizados de Wilcoxon, que é não paramétrico e não pressupõe normalidade, além de reportar a diferença média pareada com intervalo de confiança e a contagem de vitórias de cada força. Como há quinze testes, um por instância, aplica-se a correção de Holm aos p -valores para controlar a taxa de erro familiar. Comparações entre perfis ou entre localidades não são pareadas, porque envolvem instâncias diferentes com linhas de base diferentes, e por isso são descritas apenas de forma descritiva, na escala de melhoria porcentual.

3.3 Resultados descritivos

A Tabela 4 apresenta, para cada instância e força de perturbação, a linha de base, a média e o desvio padrão do valor final, a melhoria porcentual média

sobre a base, o coeficiente de variação, a média de avaliações e a média de tempo. Em todas as trinta execuções, todas as 100 sementes terminaram com valor acima da linha de base, ou seja, a fração de sementes que melhoraram a base foi de 100% em todos os casos.

Tabela 4: Estatísticas descritivas por instância e força de perturbação (s). Baseline e valores finais em soma de IQC; $\Delta\text{IQC}\%$ é a melhoria média sobre a base; $\text{CV}\%$ é o coeficiente de variação; tempo em segundos.

Localidade	Perfil	s	Base	Média	Desvio	$\Delta\text{IQC}\%$	$\text{CV}\%$	Aval. méd.
av_paulista	athlete	2	54,00	59,04	0,436	9,33	0,74	30159
av_paulista	athlete	10	54,00	58,27	0,406	7,91	0,70	22120
av_paulista	average_adult	2	47,06	51,08	0,307	8,56	0,60	30157
av_paulista	average_adult	10	47,06	50,54	0,288	7,41	0,57	22418
av_paulista	elderly	2	38,68	42,12	0,211	8,90	0,50	30148
av_paulista	elderly	10	38,68	41,60	0,229	7,56	0,55	22655
itajuba_centro	athlete	2	30,23	49,60	1,313	64,07	2,65	29696
itajuba_centro	athlete	10	30,23	46,27	1,110	53,05	2,40	16466
itajuba_centro	average_adult	2	25,41	43,55	1,578	71,36	3,62	30013
itajuba_centro	average_adult	10	25,41	40,20	1,137	58,20	2,83	17672
itajuba_centro	elderly	2	22,75	35,67	1,376	56,75	3,86	30014
itajuba_centro	elderly	10	22,75	33,26	1,051	46,19	3,16	17772
metro_ana_rosa	athlete	2	42,54	50,22	0,761	18,06	1,52	30176
metro_ana_rosa	athlete	10	42,54	49,14	0,678	15,52	1,38	21705
metro_ana_rosa	average_adult	2	38,53	44,65	0,444	15,87	0,99	30150
metro_ana_rosa	average_adult	10	38,53	43,78	0,406	13,61	0,93	20934
metro_ana_rosa	elderly	2	32,45	36,56	0,261	12,67	0,71	30129
metro_ana_rosa	elderly	10	32,45	35,92	0,243	10,71	0,68	21135
milao_italia	athlete	2	62,95	65,30	0,218	3,74	0,33	30132
milao_italia	athlete	10	62,95	64,83	0,172	3,00	0,27	22956
milao_italia	average_adult	2	57,45	59,62	0,201	3,78	0,34	30136
milao_italia	average_adult	10	57,45	59,22	0,180	3,09	0,30	23897
milao_italia	elderly	2	49,55	51,34	0,146	3,62	0,29	30156
milao_italia	elderly	10	49,55	51,10	0,135	3,12	0,27	25308
shinjuku_toquio	athlete	2	57,10	60,03	0,308	5,13	0,51	30122
shinjuku_toquio	athlete	10	57,10	59,61	0,271	4,40	0,46	21594
shinjuku_toquio	average_adult	2	51,27	53,95	0,290	5,22	0,54	30166
shinjuku_toquio	average_adult	10	51,27	53,56	0,256	4,47	0,48	22461

Localidade	Perfil	s	Base	Média	Desvio	$\Delta IQC\%$	CV%	Aval. méd.
shinjuku_toquio	elderly	2	42,23	44,39	0,262	5,11	0,59	30162
shinjuku_toquio	elderly	10	42,23	44,08	0,247	4,39	0,56	25038

Dois padrões aparecem na tabela. Primeiro, a melhoria porcentual sobre a base depende muito da instância, e essa dependência é explicada pelo valor da base: onde a base é baixa, como em **itajuba_centro**, a melhoria é grande (da ordem de 57 a 71%); onde a base já é alta, como em **milao_italia**, a melhoria é pequena (da ordem de 3 a 4%). Isso é esperado, porque há menos espaço para crescer quando o índice já parte de um valor alto. Segundo, a dispersão relativa é baixa: o coeficiente de variação fica abaixo de 4% em todas as execuções e abaixo de 1% na maioria, o que indica que o algoritmo é estável entre sementes, ou seja, sementes diferentes chegam a valores finais próximos.

3.4 Efeito da força de perturbação

A Tabela 5 apresenta a comparação pareada entre as forças 2 e 10. A diferença média é sempre negativa (força 10 menos força 2), o que significa que a força 2 produziu valores finais maiores em todas as quinze instâncias. As contagens de vitórias são quase sempre de 100 a 0 a favor da força 2; apenas em duas instâncias de **itajuba_centro** a força 10 venceu em poucas sementes (três e duas de cem). O teste de Wilcoxon rejeita a hipótese de igualdade em todas as instâncias com p -valor da ordem de 10^{-18} , e a correção de Holm mantém todas as quinze diferenças significativas ao nível de 5%. Vale observar que o p -valor de $\approx 3,9 \times 10^{-18}$ que se repete em várias linhas é o menor valor possível do teste para $n = 100$ quando todas as diferenças têm o mesmo sinal, o que ocorre nas instâncias com placar de 100 a 0.

A conclusão é direta: com este conjunto de instâncias e este orçamento, a força de perturbação menor foi melhor em todos os casos. Na escala de melhoria porcentual agregada sobre as quinze instâncias, a força 2 obteve média de 19,5% e mediana de 8,9%, contra média de 16,2% e mediana de 7,6% da força 10. A diferença entre a média e a mediana, e o desvio padrão alto (da ordem de 23 pontos percentuais), refletem a assimetria causada pelas instâncias de **itajuba_centro**, cujas melhorias percentuais muito grandes puxam a média para cima; por isso a mediana é uma medida mais representativa do caso típico.

Esse resultado é coerente com a teoria da Seção 1. Uma perturbação grande demais aproxima o ILS do random restart e reduz a chance de encontrar boas soluções, enquanto uma perturbação pequena mantém a caminhada produtiva. A recomendação de preferir várias perturbações pequenas a uma grande também vai nessa direção. O comportamento adaptativo reforça a leitura: partindo da força base 2, a força final média ficou em torno de 4,4

Tabela 5: Comparação pareada entre força 10 e força 2 (mesmas sementes). Diferença média = $f(s=10) - f(s=2)$; vitórias contam sementes em que cada força foi melhor; p é o teste de Wilcoxon; todas as diferenças são significativas após correção de Holm.

Localidade	Perfil	Dif. média	Vit. $s=2$	Vit. $s=10$	p (Wilcoxon)
av_paulista	athlete	-0,769	100	0	$3,9 \times 10^{-18}$
av_paulista	average_adult	-0,539	100	0	$3,9 \times 10^{-18}$
av_paulista	elderly	-0,520	100	0	$3,9 \times 10^{-18}$
itajuba_centro	athlete	-3,330	97	3	$5,4 \times 10^{-18}$
itajuba_centro	average_adult	-3,345	98	2	$4,8 \times 10^{-18}$
itajuba_centro	elderly	-2,404	100	0	$3,9 \times 10^{-18}$
metro_ana_rosa	athlete	-1,078	100	0	$3,9 \times 10^{-18}$
metro_ana_rosa	average_adult	-0,872	100	0	$3,9 \times 10^{-18}$
metro_ana_rosa	elderly	-0,636	100	0	$3,9 \times 10^{-18}$
milao_italia	athlete	-0,468	100	0	$3,9 \times 10^{-18}$
milao_italia	average_adult	-0,401	100	0	$3,9 \times 10^{-18}$
milao_italia	elderly	-0,244	100	0	$3,9 \times 10^{-18}$
shinjuku_toquio	athlete	-0,416	100	0	$3,9 \times 10^{-18}$
shinjuku_toquio	average_adult	-0,384	100	0	$3,9 \times 10^{-18}$
shinjuku_toquio	elderly	-0,305	100	0	$3,9 \times 10^{-18}$

(máximo 10), ou seja, o algoritmo só aumentou a força quando estagnou; partindo da base 10, a força final média subiu para cerca de 17 (máximo 18), o que agrava o problema em vez de corrigi-lo. A Figura 3 mostra o efeito no nível das distribuições: em todas as instâncias, a caixa da força 2 fica acima da caixa da força 10, com sobreposição pequena ou nula.

3.5 Critério de parada e custo computacional

A distribuição dos motivos de parada explica o mecanismo por trás da diferença. A Tabela 6 mostra que, com força 2, quase todas as execuções (1461 de 1500) usaram o orçamento inteiro de avaliações, sinal de que a caminhada continuava melhorando; com força 10, a maioria (1237 de 1500) parou por estagnação, ao atingir 80 iterações sem melhora. Em outras palavras, a força maior faz a busca estacionar antes, gastando menos avaliações e menos tempo, mas em um patamar pior. Isso se reflete no custo: a média de avaliações da força 2 fica próxima do teto ($\approx 30\,000$), enquanto a da força 10 varia entre cerca de 16 000 e 25 000; e o tempo médio da força 2 é maior, na mesma proporção.

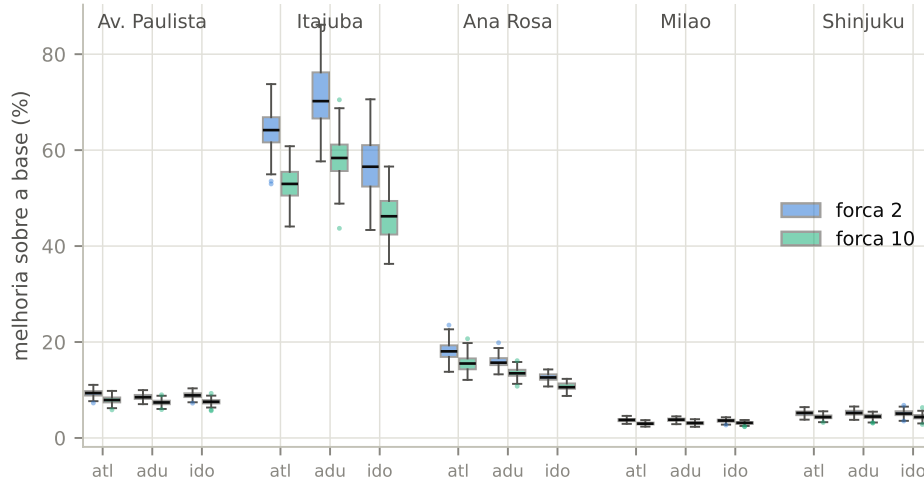


Figura 3: Distribuição, entre as 100 sementes, da melhoria porcentual sobre a linha de base, por instância e força de perturbação. Caixas indicam quartis, a linha central é a mediana, e os pontos isolados são valores fora de 1,5 vez o intervalo interquartilico.

Tabela 6: Motivos de parada, contagem sobre as 1500 execuções de cada força.

Motivo de parada	Força 2	Força 10
Orçamento de avaliações esgotado (<code>max_evaluations</code>)	1461	263
Estagnação (<code>max_no_improve</code>)	39	1237

3.6 Convergência

Para observar a convergência, a Tabela 7 mostra, em duas instâncias representativas, a média sobre as sementes do melhor valor alcançado até certos números de avaliações, e a fração da melhoria total já obtida em cada marco. As duas instâncias são `itajuba_centro/average_adult`, de melhoria grande, e `milao_italia/elderly`, de melhoria pequena. Em ambas, a maior parte do ganho ocorre cedo: por volta de 5 000 avaliações já se alcança perto de 90% da melhoria total, e as avaliações restantes rendem ganhos pequenos. A força 10 chega mais rápido a uma fração maior do seu próprio ganho, mas o seu patamar final é mais baixo do que o da força 2, o que é consistente com a tabela descritiva.

A Figura 4 complementa a tabela com as curvas completas de convergência, em escala logarítmica de avaliações, mostrando a média e a faixa interquartilica sobre as sementes.

Tabela 7: Convergência: média do melhor valor por número de avaliações, e fração da melhoria total (entre parênteses).

Instância / força	1.000	5.000	10.000	20.000	30.000
itajuba avg. ($s=2$)	38,20 (70%)	41,34 (88%)	42,32 (93%)	43,20 (98%)	43,54 (100%)
itajuba avg. ($s=10$)	36,83 (77%)	39,09 (93%)	39,84 (98%)	40,16 (100%)	40,20 (100%)
milao eld. ($s=2$)	50,89 (75%)	51,19 (92%)	51,26 (96%)	51,32 (99%)	51,34 (100%)
milao eld. ($s=10$)	50,77 (79%)	50,97 (92%)	51,04 (96%)	51,08 (99%)	51,10 (100%)

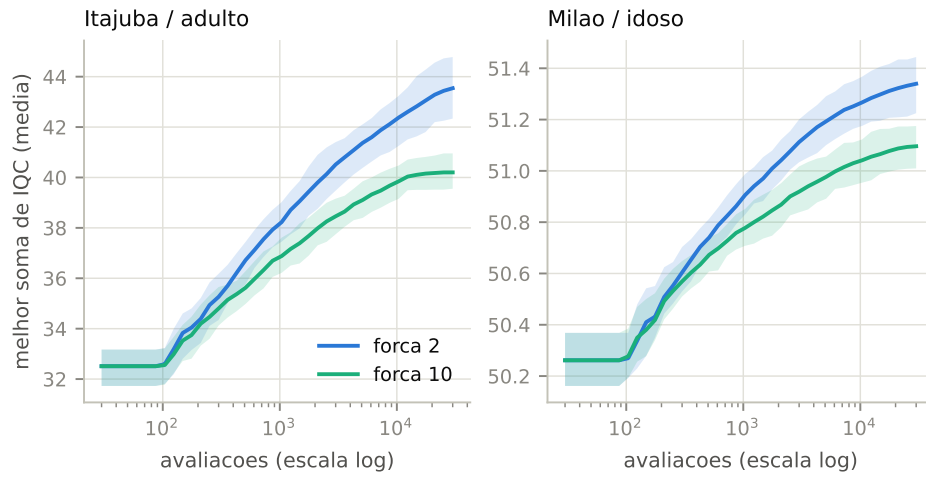


Figura 4: Convergência do ILS: média, sobre as 100 sementes, do melhor valor em função do número de avaliações (linhas) e faixas interquartílicas (áreas sombreadas), para as duas forças de perturbação. A força 2 satura em um patamar mais alto nas duas instâncias.

3.7 Influência da solução inicial

A teoria da Seção 1 prevê que, em execuções longas, a memória da solução inicial se dissipa se a caminhada explora bem o espaço dos ótimos locais. Isso é testável nos dados: se a memória persiste, sementes cujo ponto de partida aleatório é melhor deveriam terminar melhor. A associação foi medida pela correlação de Spearman entre o valor da solução inicial (antes da busca local) e o melhor valor final da mesma execução, por instância, nas execuções com força 2.

As correlações são todas positivas, mas fracas: variam de +0,06 a +0,50, e apenas três das quinze instâncias permanecem significativas ao nível de 5% após a correção de Holm para os quinze testes (*av_paulista/elderly*, $\rho_s = 0,50$; *metro_ana_rosa/athlete*, $\rho_s = 0,41$; *av_paulista/average_adult*, $\rho_s = 0,40$). Mesmo nesses casos, a fração de variação explicada é modesta.

A Figura 5 contrasta uma instância com associação detectável e outra sem: em ambas, a nuvem é larga, e partidas ruins frequentemente terminam entre as melhores. A leitura é a prevista pela teoria: com 30 000 avaliações de orçamento, a caminhada explora o suficiente para apagar a maior parte da influência do ponto de partida, o que também justifica a escolha de uma solução inicial aleatória simples em vez de uma construção gulosa.

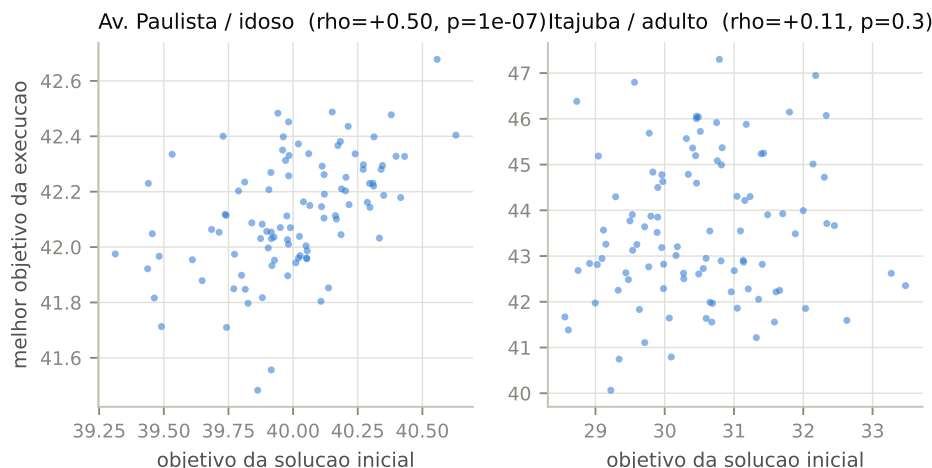


Figura 5: Melhor valor da execução contra o valor da solução inicial aleatória, por semente (força 2). À esquerda, a instância com a maior correlação da análise; à direita, uma instância típica, sem associação significativa.

3.8 Distribuição do IQC por hexágono: assimetria e curtose

As análises anteriores olham para a soma do IQC. Esta subseção olha para a forma da distribuição do IQC entre os hexágonos, antes e depois da intervenção, aplicando a melhor alocação global de cada instância (força 2) e recalculando o IQC de todos os hexágonos. A assimetria (g_1) e o excesso de curtose (g_2) amostrais são acompanhados dos testes de D'Agostino; a Tabela 8 traz os valores e as Figuras 6 e 7 os ilustram.

Antes da intervenção, doze das quinze instâncias têm assimetria positiva significativa ao nível de 5% (g_1 entre +0,45 e +1,37): a massa concentra-se nos hexágonos de IQC baixo, com uma cauda de poucos hexágonos bons. Depois da melhor alocação do ILS, a assimetria cai em treze das quinze instâncias e permanece significativa em oito; nos casos de queda mais forte, como Itajubá, o g_1 passa de +0,82–+1,08 para +0,05–+0,10 nos perfis atleta e adulto. O excesso de curtose também se desloca para baixo na maioria das instâncias, com os pontos migrando para a região de distribuições mais simétricas e mais achatadas no plano (g_1, g_2). Milão, cuja distribuição já era simétrica antes, muda pouco. A leitura substantiva é a mesma nos dois

sentidos: a otimização não apenas eleva a soma do IQC, ela desconcentra a distribuição, elevando preferencialmente a cauda baixa — consequência de o objetivo ser a soma sob normalização min-max, em que elevar hexágonos ruins rende mais do que reforçar hexágonos já bons.

Tabela 8: Assimetria (g_1) e excesso de curtose (g_2) da distribuição do IQC por hexágono, antes e depois da melhor alocação do ILS (força 2). O símbolo † marca valores significativamente diferentes de zero pelo teste de D’Agostino ao nível de 5%.

Localidade	Perfil	g_1 antes	g_1 depois	g_2 antes	g_2 depois
av_paulista	athlete	+0,45 †	+0,29	-0,37	-0,35
av_paulista	average_adult	+0,52 †	+0,46 †	-0,24	-0,01
av_paulista	elderly	+0,58 †	+0,60 †	-0,07	+0,17
itajuba_centro	athlete	+0,82 †	+0,10	-0,34	-0,99 †
itajuba_centro	average_adult	+1,08 †	+0,05	+0,34	-0,99 †
itajuba_centro	elderly	+1,37 †	+0,54 †	+1,48 †	-0,42
metro_ana_rosa	athlete	+0,76 †	+0,21	+0,33	-0,79 †
metro_ana_rosa	average_adult	+0,92 †	+0,47 †	+0,85	-0,54
metro_ana_rosa	elderly	+1,01 †	+0,86 †	+1,03 †	+0,56
milao_italia	athlete	-0,09	-0,11	-0,93 †	-0,95 †
milao_italia	average_adult	+0,01	-0,03	-0,84 †	-0,88 †
milao_italia	elderly	+0,13	+0,09	-0,62 †	-0,73 †
shinjuku_toquio	athlete	+0,63 †	+0,45 †	-0,08	-0,27
shinjuku_toquio	average_adult	+0,77 †	+0,60 †	+0,33	+0,07
shinjuku_toquio	elderly	+0,88 †	+0,71 †	+0,51	+0,12

3.9 Síntese

Os dados sustentam cinco conclusões. Primeiro, o ILS melhora a linha de base de forma consistente: em todas as quinze instâncias e todas as 100 sementes, o valor final ficou acima da base, e a dispersão entre sementes é baixa, o que indica robustez. Segundo, a magnitude da melhoria depende da instância e, em particular, do valor da linha de base, com ganhos grandes onde a base é baixa e ganhos pequenos onde a base já é alta. Terceiro, a força de perturbação tem efeito claro e estatisticamente significativo: a força menor venceu a maior em todas as instâncias, com o comportamento adaptativo e a distribuição dos motivos de parada mostrando que a força maior leva à estagnação precoce — exatamente o que a teoria prevê sobre

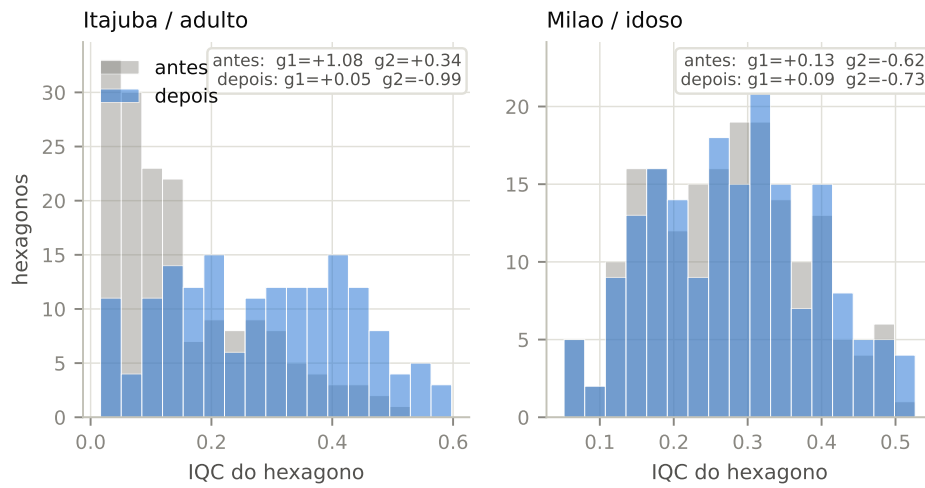


Figura 6: Distribuição do IQC por hexágono antes (cinza) e depois (azul) da melhor alocação do ILS, nas duas instâncias representativas, com g_1 e g_2 anotados.

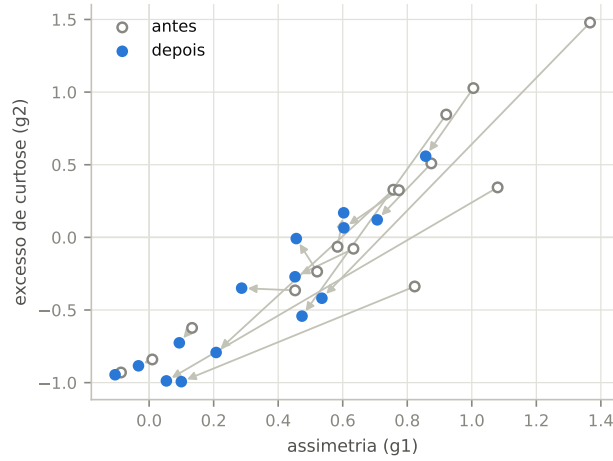


Figura 7: Migração das quinze instâncias no plano assimetria $\times$ excesso de curtose, da distribuição de base (círculos vazios) para a distribuição após a melhor alocação do ILS (círculos cheios).

perturbações grandes aproximarem o método do random restart. Quarto, a influência da solução inicial sobre o resultado final é fraca: a correlação entre partida e chegada só é detectável em três instâncias e é modesta mesmo nelas, confirmando que a caminhada dissipa a memória do ponto de partida no orçamento usado. Quinto, além de aumentar a soma do IQC, a intervenção torna a distribuição do IQC entre hexágonos mais simétrica e menos concentrada — a assimetria positiva significativa presente em doze instâncias antes cai em treze delas depois —, ou seja, a melhora se espalha pela região em vez de se restringir a poucos hexágonos.

Referências

- [1] H. R. Lourenço, O. C. Martin e T. Stützle. Iterated Local Search. In F. Glover e G. Kochenberger (eds.), *Handbook of Metaheuristics*, cap. 11, pp. 321–353. Kluwer Academic Publishers, 2003.